CSSG EDUCATION TEAM

System design: how to tackle any problem

Worked example: a guestbook for your portfolio

Frontend Backend Database

QUICK RECAP

Three layers, every app

Frontend

What the user sees and touches. Runs in *their* browser.

Backend

The rules. Runs on a server *you* control.

Database

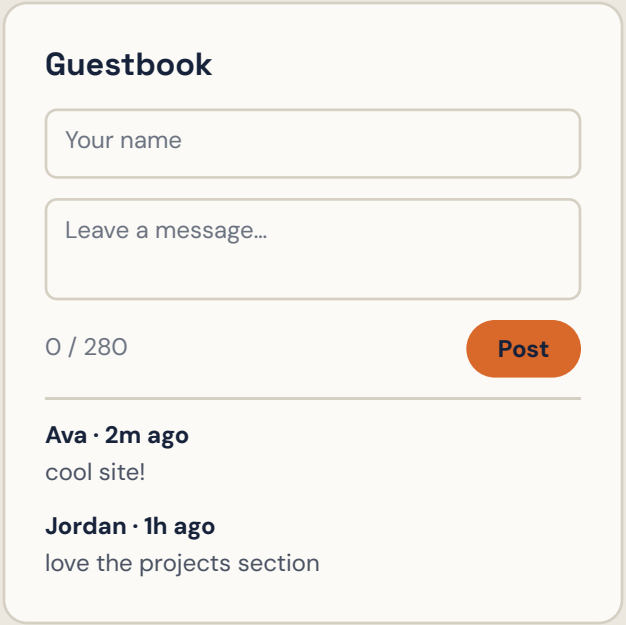
The memory. Only the backend talks to it.

Today: apply these to one real problem, step by step.

THE PROBLEM

Visitors can leave a message on your portfolio. Everyone sees the newest 20.

Before designing anything: what has to be stored? Who can write? What if someone spams it?



THE FRAMEWORK

Five questions for any system

1	What does the user need to do?	Requirements
2	What has to be true while they do it?	Constraints
3	What does the system remember?	Data
4	Where does each piece of work happen?	Layers
5	What crosses each arrow?	Communication

QUESTION 1 OF 5 · REQUIREMENTS

What does the user need to do?

Action 1

Leave a message

Action 2

See recent messages

Write actions as verbs. If it isn’t on this list, we don’t build it yet.

QUESTION 2 OF 5 · CONSTRAINTS

What has to be true while they do it?

Messages are short

280 characters max; names up to 50.

Anyone can post

No login, so we can’t trust who’s sending.

Spam is limited

One person can’t post 100 times a minute.

Newest first

Visitors see the latest 20, quickly.

QUESTION 3 OF 5 · DATA

What does the system remember?

One entry (a document)

```
_id: ObjectId("6531…")        System sets
name: "Ava"                   User sets
message: "cool site!"         User sets
createdAt: Oct 28, 3:14pm     System sets
```

One noun: an entry. In NoSQL, each one is stored as a document — a JSON-like object.

Every entry gets a unique ID the system creates. In MongoDB, that's an ObjectId.

Ask: why shouldn't the user pick the ID or the time?

QUESTION 4 OF 5 · LAYERS

Where does each piece of work happen?

Frontend, backend, or database? Sort these with the person next to you.

Count characters as the user types	Reject messages over 280 characters	Keep every entry permanently
Limit how often one person can post	Show the new message instantly	Stamp the ID and time posted
Return the newest 20, fast	Disable Post when the box is empty	Hide private fields from visitors

QUESTION 4 OF 5 · LAYERS

Each job has a home

Frontend

- Count characters as you type
- Disable Post when empty
- Show the new message instantly

Backend

- Reject messages over 280
- Limit how often one person posts
- Stamp the ID and time
- Hide private fields

Database

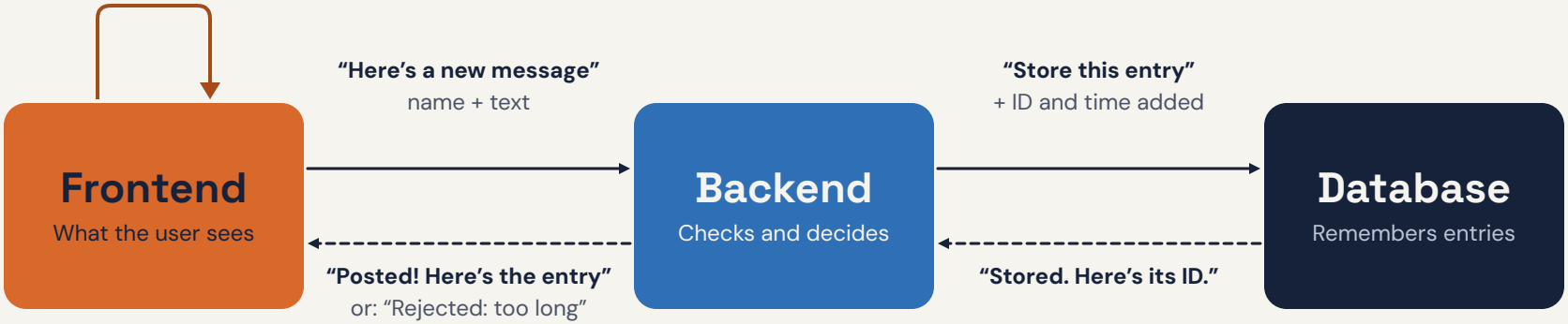
- Keep every entry
- Return the newest 20, fast

Rule: if it protects the system, it can’t live only on the frontend. Anyone can skip your browser.

QUESTION 5 OF 5 · POSTING A MESSAGE

What crosses each arrow?

Client-side only (never leaves the browser): count characters, block an empty post, show it instantly

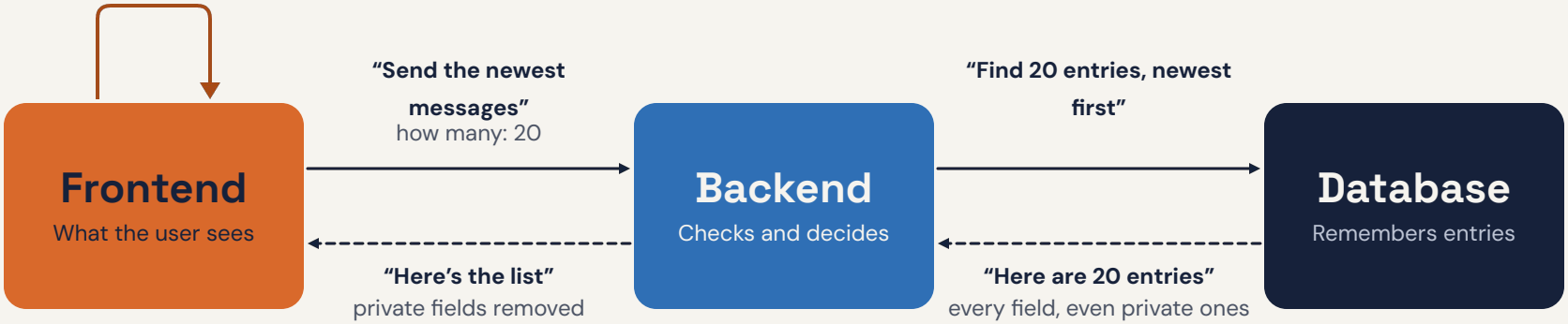


Solid = request, dashed = response. **The frontend asks, the backend decides, the database remembers.**

QUESTION 5 OF 5 · LOADING THE LIST

Same arrows, new contents

Client-side only: show a loading state while waiting, then render each entry as a card

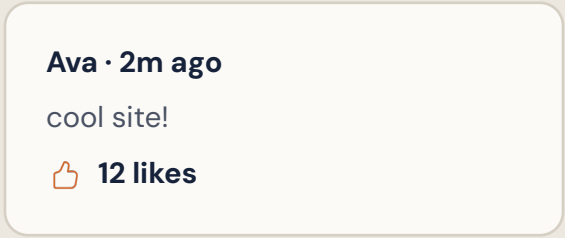


Notice the shape didn't change. **Only what crosses the arrows did.**

YOUR TURN · 5 MINUTES

Add a like button to each entry

1. What does the user need to do?
2. What has to be true? Can someone like twice?
3. What does the system remember? The hard one.
4. Where does each piece of work happen?
5. What crosses each arrow?



DEBRIEF · QUESTION 3

Where does a like live?

A number on the entry

`likes: 12`

Good: dead simple, one field.

Bad: can't tell who liked, so it can't stop double-likes.

A list inside the entry

`likedBy: [ … ]`

Good: blocks duplicates.

Bad: the list grows forever inside every entry.

Its own collection

`one document per like`

Good: clean, no duplicates, no bloat.

Bad: showing the count takes extra work.

No single right answer. In NoSQL this choice is called **embedding vs. referencing** — more in the backend class.

TAKEAWAY

System design is five questions, answered in order.

1 Requirements | 2 Constraints | 3 Data | 4 Layers | 5 Arrows

Next up: public APIs — what happens when the data comes from someone else's server.